

CA-Mem: Overcoming the Cross-Lingual Knowledge Bottleneck via Concept-Anchored Memory Agent

Anonymous ACL submission

Abstract

Large language models often degrade sharply when reasoning in low-resource languages. We attribute this degradation to a cross-lingual knowledge bottleneck: fragile LRL surface representations do not reliably elicit knowledge associated with dense, high-resource language conceptual hubs. Existing prompting and translation strategies either assume correct initial knowledge routing or preserve task-irrelevant surface details, which can amplify reasoning drift. **CA-Mem** is a Concept-Anchored Memory Agent that separates multilingual reasoning into three stages. First, *Concept Anchoring* abstracts the LRL query into an HRL concept and boundary definition, providing a more stable semantic interface. Second, CA-Mem retrieves from a curated *Concept Usage Memory* that stores procedural heuristics distilled from source-side problem-solving trajectories rather than raw facts or full QA traces. Third, the model answers the original query with both the concept anchor and the retrieved usage guidance. On Global-MMLU, CA-Mem achieves 66.24 macro accuracy and improves over strong translation and structured-prompting baselines by more than 4 points. The gains further generalize to INCLUDE and MGSM, as well as across Qwen, Llama, Ministral, and Gemma, suggesting that concept-anchored procedural memory provides an effective interface in the evaluated low-resource multilingual reasoning settings.

1 Introduction

Large language models (LLMs) demonstrate strong reasoning and knowledge-acquisition capabilities, yet they still degrade substantially when applied to low-resource languages (LRLs) (Qin et al., 2024; Zhang et al., 2025b). This discrepancy is closely related to the imbalanced distribution of pre-training corpora. High-resource languages (HRLs), such as English, provide much denser training signals, allowing LLMs to form more stable latent concept

representations. In contrast, sparse exposure to LRLs can produce less reliable representations that fail to align with these established conceptual hubs (Zhu et al., 2024; Cao et al., 2024). We follow prior work in referring to this as the cross-lingual knowledge bottleneck (Goworek et al., 2025): LRL queries may trigger unstable routing paths in latent space, causing reasoning failures before downstream inference can use the relevant knowledge.

To address this bottleneck, conventional approaches primarily rely on prompt formatting or surface-level language conversion. Reasoning enhancement techniques, such as Chain-of-Thought (CoT) (Wei et al., 2022) and its structured variants (Qi et al., 2025), enforce step-by-step generation. However, they lack mechanisms to ensure correct initial concept activation; if the model misroutes the LRL query at the outset, structural prompting may simply elaborate an incorrect trajectory (Chen et al., 2025). Alternatively, translation-based strategies, such as Translate-Test (Etxaniz et al., 2024) or Self-Translate (Liu et al., 2025a), attempt to bypass the LRL barrier by converting the entire prompt into an HRL. While this can mitigate surface-level comprehension issues, holistic translation may preserve task-irrelevant details and noisy distractors. It fails to explicitly decouple and highlight the core concepts required for problem-solving, leaving the model vulnerable to reasoning drift and context interference.

Recent work explores memory-augmented agents that provide external knowledge or historical reasoning trajectories (Liu et al., 2025b; Zhang et al., 2025a). MemoryBank (Zhong et al., 2024), A-Mem (Xu et al., 2026), and Mem0 (Chhikara et al., 2025) utilize Retrieval-Augmented Generation (RAG) pipelines to fetch relevant past interactions or factual paragraphs. Despite their success in monolingual or HRL settings, applying these memory systems directly to LRL reasoning introduces new challenges. First, querying a memory bank us-

ing raw LRL text or its direct translation can suffer from cross-lingual representation mismatch, leading to noisy or redundant similarity retrieval (Hu et al., 2026). Second, retrieving raw factual texts or complete historical QA pairs merely provides declarative context. It does not directly equip the model with the procedural knowledge of *how* to apply a specific concept under complex constraints, and it can exacerbate lost-in-the-middle effects or token inefficiency (Liu et al., 2026b).

To tackle these challenges, we propose CAMem, a framework that shifts from passive text retrieval to explicit conceptual alignment and procedural experience reuse. Our approach operates via a three-stage mechanism. First, the agent executes concept anchoring, where it abstracts the raw LRL query into a concise HRL concept name and its boundary definition. Second, using this explicit concept anchor, the agent retrieves from a curated concept usage memory. Third, the agent conducts memory-augmented reasoning by combining the original query, the activated concept-description pair, and the retrieved usage memory. Instead of storing raw texts, this memory retains procedural summaries of how specific concepts were successfully applied in source-side examples. Experiments on multiple multilingual benchmarks across diverse LLMs show that our framework consistently outperforms competitive translation and structural baselines. In summary, our main contributions are as follows:

- We propose the concept-anchored memory agent framework. By explicitly generating HRL concepts and definitions, we align fragile LRL inputs into a stable, high-resource concept space, improving conceptual routing.
- We introduce the concept usage memory to store and retrieve procedural problem-solving experiences rather than declarative texts.
- We empirically show that providing concept-aligned usage experience improves multilingual reasoning and generalizes across multiple LLM families.

2 Related Work

Low-Resource Multilingual Reasoning LLM reasoning capabilities remain skewed toward HRLs (Huang et al., 2023; Zhu et al., 2024), causing large performance drops on LRL benchmarks (Shi et al.,

2022; Ponti et al., 2020). Representation analyses suggest that LRL inputs often map to less stable regions in latent space, making them harder to align with HRL concept hubs (Cao et al., 2024; Tang et al., 2024). Consequently, an important bottleneck in LRL reasoning is knowledge elicitation rather than only knowledge absence (Qin et al., 2024). Our work addresses this bottleneck by explicitly mapping fragile LRL inputs into a high-resource concept space before reasoning.

Translation and Prompting Methods Training-free interventions typically mitigate LRL deficits through language conversion or reasoning formatting. Conversion strategies, such as English-centric reasoning (Etxaniz et al., 2024) and Self-Translate (Liu et al., 2025a), map LRL queries to English before inference. However, as demonstrated by Liu et al. (2025a), holistic translation can preserve surface-level noise and task-irrelevant distractors, failing to isolate the core concepts necessary for deduction. Alternatively, prompting paradigms like CoT (Wei et al., 2022) and SoT (Qi et al., 2025) enforce step-by-step rationales. Recent multilingual reasoning studies also show that models can generate English rationales for non-English questions, motivating cross-lingual CoT-style baselines (Ranaldi et al., 2025). Yet, these methods implicitly assume correct initial knowledge routing. If an LRL input activates the wrong concept, structural prompting can elaborate an already-misaligned trajectory (Chen et al., 2025). Our Concept Anchoring operates orthogonally: it abstracts the LRL input into a concise HRL concept definition before any structural reasoning is applied.

Retrieval and Memory-Augmented Reasoning RAG fetches external contexts to reduce hallucinations (Gao et al., 2023), but applying RAG to LRLs suffers from cross-lingual representation mismatch, yielding noisy and redundant contexts (Zhang et al., 2025b). To improve retrieval efficiency, agent memory systems structure historical interactions (Zhong et al., 2024; Chhikara et al., 2025). Recent architectural optimizations, such as SimpleMem (Liu et al., 2026b) and Beyond RAG (Hu et al., 2026), decouple and compress episodic traces to prevent similarity collapse. Despite these structural advances, existing memories predominantly retrieve declarative facts or raw historical states. They lack the procedural heuristics required to apply specific concepts to novel scenarios (Kim et al., 2024). Building upon hierarchical memory distillation, we introduce the concept usage

Offline Memory Curation

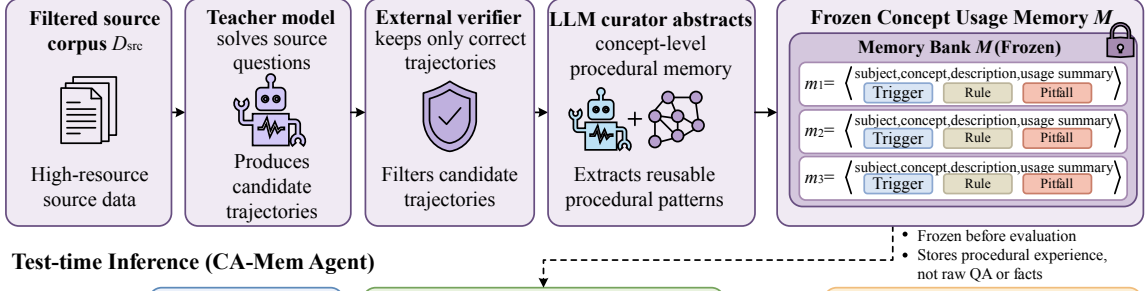


Figure 1: **CA-Mem**. The agent first anchors a multilingual input into an English concept-description pair, retrieves concept-aligned procedural memory, and then performs memory-augmented reasoning.

memory. Instead of caching declarative texts, our memory stores abstract, procedural usage experiences. Retrieving these experiential guidelines via anchored HRL concepts reduces cross-lingual mismatch and injects procedural knowledge for LRL reasoning.

3 Methodology

We propose CA-Mem, a structured framework designed to mitigate the cross-lingual knowledge bottleneck in LLMs. Rather than relying on fragile surface-level predictions from LRLs, CA-Mem decomposes inference into three sequential phases: Concept Anchoring, Cognitive Experience Retrieval, and Memory-Augmented Reasoning. As illustrated in Figure 1, the agent first abstracts the raw LRL query into an HRL conceptual representation, uses this semantic anchor to retrieve relevant procedural experiences from a frozen memory bank, and then combines these elements to predict the final answer.

3.1 Concept Anchoring for Cross-Lingual Elicitation

Directly prompting an LLM with an LRL query x can force reasoning from a sparse and less reliable representation, leading to knowledge elicitation failures. To address this, Concept Anchoring projects the LRL input onto a denser HRL conceptual interface before deduction. Given an activation instruction $\mathcal{I}_{act}$, the agent decodes an HRL concep-

tual tuple $z = (c, d)$, where c is the core concept name and d is its boundary definition. Rather than performing an intractable joint search over the concept and description spaces, we enforce a directed dependency during inference. We obtain the tuple with sequential greedy decoding (temperature $\tau = 0$):

$$c^* = \arg \max_c P_\theta(c \mid x, \mathcal{I}_{act}) \quad (1)$$

$$d^* = \arg \max_d P_\theta(d \mid x, \mathcal{I}_{act}, c^*) \quad (2)$$

This autoregressive factorization makes c^* an explicit semantic anchor, while the subsequent description d^* bounds its interpretive scope. This mapping reduces LRL-specific surface noise and stabilizes the initial reasoning context.

3.2 Concept Usage Memory

To bridge the gap between concept identification and execution, we introduce the Concept Usage Memory, a structured repository designed to encode and supply procedural heuristics.

3.2.1 Memory Node Representation

Unlike standard RAG paradigms that cache declarative facts or raw question-answer pairs, our architecture isolates procedural knowledge. We formalize the memory bank as a set of structured nodes $\mathcal{M} = \{m_1, m_2, \dots, m_K\}$. Each node m_i is instantiated as a heterogeneous tuple:

$$m_i = \langle s_i, c_i, d_i, u_i \rangle \quad (3)$$

where $s_i \in \mathcal{S}$ represents the subject or domain constraint, $c_i \in \mathcal{C}$ denotes the semantic concept anchor, $d_i \in \mathcal{D}$ defines the precise conceptual boundary, and $u_i \in \mathcal{U}$ is the core usage summary. The usage summary u_i fundamentally distinguishes our approach from traditional memory stores. Rather than preserving specific factual states, u_i encodes the generalized procedural heuristics governing the application of c_i . We treat u_i as a compact set of constraints that guide the downstream reasoning function. Specifically, $u_i = \{\omega_{\text{trig}}, \omega_{\text{rule}}, \omega_{\text{err}}\}$ contains trigger invariants, decision rules, and negative error boundaries (i.e., common pitfalls), respectively. By retrieving u_i , the agent acquires explicit instructions on how to operationalize the anchored concept within the targeted domain s_i .

3.2.2 Offline Memory Curation

To provide generalizable procedural experiences without introducing evaluation bias, the offline memory bank $\mathcal{M}$ is curated from a decontaminated source corpus filtered against the evaluation benchmarks, denoted $\mathcal{D}_{\text{src}}$. Candidate memory nodes originate from verifier-approved solving trajectories. To prevent the memory from overfitting to surface variables (e.g., instance-specific entities or numbers), we distill raw historical trajectories $\mathcal{T}_{c_i}$ into generalized usage summaries u_i . Conceptually, this distillation is motivated by the Information Bottleneck (IB) principle: retaining information relevant to general solving logic while minimizing instance-specific details. Because our curator is a frozen LLM Φ_{curate} operating zero-shot, we do not explicitly optimize a parameterized mutual information objective. Instead, we operationalize this compression-relevance trade-off through constrained prompt abstraction:

$$u_i = \Phi_{\text{curate}}(\mathcal{T}_{c_i} \mid \Omega_{\text{IB}}) \quad (4)$$

where Ω_{IB} represents the explicit instruction constraints that mandate the removal of specific instance details and the retention of purely procedural heuristics (trigger invariants, decision rules, and error boundaries). Once curated, $\mathcal{M}$ remains entirely frozen during evaluation. Test queries only execute read operations, so downstream gains reflect zero-shot transfer of concept-aligned procedural knowledge under our filtering protocol rather than test-set updates.

3.3 Memory-Augmented Retrieval and Reasoning

After constructing a conceptual anchor and an offline procedural memory bank, the framework integrates these elements at inference time. This design avoids direct retrieval with raw LRL queries, which can suffer from cross-lingual representation mismatch. Inference therefore proceeds through Cognitive Experience Retrieval followed by Memory-Augmented Reasoning.

3.3.1 Cognitive Experience Retrieval

To fetch procedural heuristics, we implement a concept-based retrieval mechanism. The retrieval query concatenates the domain subject s , the generated concept c^* , and the generated description d^* , isolating retrieval from the syntactic noise of the original LRL input x . Let $\mathcal{E}_\phi : \mathcal{V}^* \rightarrow \mathbb{R}^d$ denote a dense embedding encoder. We formulate the retrieval query vector $\mathbf{q} \in \mathbb{R}^d$ as the encoded representation of the concatenated conceptual anchor:

$$\mathbf{q} = \mathcal{E}_\phi(\text{concat}(s, c^*, d^*)) \quad (5)$$

Similarly, for each candidate node $m_i \in \mathcal{M}$, we compute a key vector $\mathbf{k}_i = \mathcal{E}_\phi(\text{concat}(s_i, c_i, d_i))$. The relevance of a memory node is quantified via cosine similarity $S_{\text{cos}}(\mathbf{q}, \mathbf{k}_i)$. To reduce the injection of irrelevant or mismatched procedural memory, we enforce a gated retrieval strategy. The top-ranked memory node is used only when its similarity exceeds a predefined confidence threshold γ . The retrieved memory m^* is defined as:

$$\hat{i} = \arg \max_{i \in \{1, \dots, K\}} S_{\text{cos}}(\mathbf{q}, \mathbf{k}_i) \quad (6)$$

$$m^* = \begin{cases} m_{\hat{i}}, & \text{if } S_{\text{cos}}(\mathbf{q}, \mathbf{k}_{\hat{i}}) \geq \gamma \\ \emptyset, & \text{otherwise} \end{cases} \quad (7)$$

When $m^* = \emptyset$, the framework defaults to the base concept-anchored setting without external memory injection. This thresholding mechanism acts as a semantic filter, keeping the injected procedural heuristic u^* closely aligned with the activated concept c^* .

3.3.2 Memory-Augmented Reasoning

In the final reasoning phase, the agent synthesizes the raw LRL input x , the activated conceptual tuple (c^*, d^*) , and the retrieved usage summary u^* (if $m^* \neq \emptyset$). The final answer A^* is decoded by maximizing the conditional likelihood:

$$A^* = \arg \max_A \prod_{t=1}^{|A|} P_\theta(a_t \mid x, c^*, d^*, u^*, a_{<t}) \quad (8)$$

Directly decoding from a sparse LRL input yields a highly uncertain prediction space. By conditioning generation on c^* and u^* , we introduce explicit priors. From an information-theoretic perspective, informative conditioning can reduce the effective prediction space when the added variables are relevant. Practically, the concept c^* aligns decoding with the stable HRL manifold, while u^* acts as a procedural regularizer that penalizes rule violations. This conditioning can reduce reasoning drift and conceptual misapplication during execution.

4 Experiments

We evaluate CA-Mem on three multilingual benchmarks: Global-MMLU (Singh et al., 2025), INCLUDE (Romanou et al., 2025), and MGSM (Shi et al., 2022). For Global-MMLU, we report per-language accuracy and Macro Avg. on bn, sw, te, ne, and hi. For INCLUDE, we report per-language accuracy and Weighted Avg. on bn, te, ne, and hi because language-specific test sizes differ. For MGSM, we report per-language accuracy and Macro Avg. on bn, sw, and te. Appendix A provides detailed language counts, including the English and Chinese auxiliary splits.

4.1 Model Variants and Data Construction

We compare three model variants: Base, Direct-SFT, and CA-SFT. Base denotes the underlying model without task-specific SFT. Direct-SFT uses answer-only supervision, whereas CA-SFT uses concept-anchored supervision with two complementary tasks: concept generation and concept-grounded answering. Both supervised data and the memory bank are derived from a decontaminated MMLU-ProX (Xuan et al., 2025) source pool with instance-level duplicate filtering. Detailed leakage controls are provided in Appendix E. For CA-SFT, we retain 4,700 training rows for each of the five training languages. For memory construction, we curate memory only from verifier-approved teacher trajectories, yielding 5,087 final concept memory nodes. We freeze the memory bank during evaluation and answer-sanitize memory entries by excluding question text, option text, answer labels, gold answers, correct option text, and answer-bearing

shortcuts. Detailed prompt templates and implementation settings are provided in Appendix D and Appendix C.

4.2 Baselines and Inference Settings

We compare Zero-shot, CoT, Cross-CoT, Self-Translate, SoT, CA-only, and CA-Mem. CA-only first generates an English concept and description, then answers without retrieval. CA-Mem augments CA-only with frozen memory retrieval using the subject, generated concept, and generated description. Full evaluation prompts are provided in Appendix D.

4.3 Implementation Summary

Unless otherwise specified, CA-Mem uses Top-1 retrieval with the development-selected threshold $\gamma = 0.8$. Memory is injected only when the retrieved node passes the retrieval cutoff; otherwise the system falls back to CA-only. Detailed hyperparameters, retrieval encoder settings, decoding rules, and answer extraction are deferred to Appendix C. Appendix H further compares CA-Mem with sanitized raw QA and raw trajectory memories, showing that curated procedural usage summaries provide stronger gains than raw retrieved contexts. Appendix G.4 further shows that CA-Mem adds only modest inference overhead over CA-only, increasing average total tokens from 1016.66 to 1046.78, while remaining substantially cheaper than SoT (1268.30 total tokens).

5 Results and Analysis

5.1 Main Results

Table 1 reports the main results on Global-MMLU. CA-Mem yields the highest accuracy across all evaluated low-resource languages. Table 5 in the appendix reports the full INCLUDE and MGSM results. Under the CA-SFT setting, CA-Mem attains a weighted average of 59.12 on INCLUDE and a macro average of 28.00 on MGSM. It outperforms the strongest structural baseline (SoT) by 4.75 and 4.00 absolute points, respectively, and exceeds the holistic translation strategy (Self-Translate) by 5.74 and 5.73 points. These margins support the efficacy of combining explicit conceptual anchoring with procedural memory retrieval for cross-lingual reasoning. Isolating the initial cognitive phase, CA-only exceeds both standard reasoning prompts and language conversion methods. On INCLUDE (CA-SFT), CA-only scores 54.92, sur-

Model	Method	bn	sw	te	ne	hi	Macro Avg.
Base	Zero-shot	52.50	52.86	43.14	48.58	54.14	50.24
	CoT	54.02	54.09	44.71	50.40	55.54	51.75
	Cross-CoT	55.10	54.58	45.20	51.48	56.51	52.57
	Self-Translate	55.60	54.66	45.55	51.42	56.23	52.69
	SoT	55.96	54.85	45.75	51.97	57.26	53.16
	CA-only	56.67	56.00	46.08	52.81	57.80	53.87
	CA-Mem	60.77	60.37	50.10	56.83	61.66	57.95
Direct-SFT	Zero-shot	55.23	52.51	48.39	52.21	56.47	52.96
	CoT	57.54	54.87	51.50	54.65	58.58	55.43
	Cross-CoT	58.42	56.10	52.98	56.10	60.12	56.74
	Self-Translate	59.17	56.95	53.55	57.26	60.75	57.54
	SoT	57.59	55.00	51.49	55.24	58.91	55.65
	CA-only	60.68	57.62	53.67	57.81	61.65	58.29
	CA-Mem	65.09	62.33	57.87	62.37	65.94	62.72
CA-SFT	Zero-shot	57.11	53.97	50.83	55.75	59.75	55.48
	CoT	60.42	57.95	54.85	58.97	63.15	59.07
	Cross-CoT	62.18	59.09	56.52	60.56	64.26	60.52
	Self-Translate	63.50	60.03	57.29	61.67	65.87	61.67
	SoT	62.80	59.56	56.67	61.52	65.56	61.22
	CA-only	63.61	60.28	58.05	61.71	66.03	61.94
	CA-Mem	67.73	64.63	62.23	66.20	70.40	66.24

Table 1: Full Global-MMLU results. CA-Mem denotes CA-only with concept usage memory. Macro Avg. is computed across the five evaluated low-resource languages.

Ablation	Configuration	bn	sw	te	ne	hi	Macro Avg.	Delta
Group 1: Retrieval Quality								
CA-only	no memory	63.61	60.28	58.05	61.71	66.03	61.94	-
Random Memory	random memory	63.47	60.19	57.96	61.64	65.87	61.82	-0.12
Wrong Memory	concept-mismatched memory	63.04	59.68	57.43	61.12	65.48	61.35	-0.59
Retrieved Memory	concept-aligned memory	67.73	64.63	62.23	66.20	70.40	66.24	+4.30
Group 2: Memory Content								
CA-only	no memory	63.61	60.28	58.05	61.71	66.03	61.94	-
Description Only	description only	64.82	61.29	59.29	62.92	67.26	63.12	+1.18
Usage Summary Only	usage summary only	66.84	63.51	61.30	64.90	69.35	65.18	+3.24
Full Memory	description + usage summary	67.73	64.63	62.23	66.20	70.40	66.24	+4.30
Group 3: Retrieval Strategy								
CA-only	no memory	63.61	60.28	58.05	61.71	66.03	61.94	-
LRL-Question Retrieval	LRL question	64.32	61.08	58.97	62.73	66.91	62.80	+0.87
English-Question Retrieval	English question	66.12	62.87	60.79	64.41	68.50	64.54	+2.60
Concept-Only Retrieval	subject + concept	66.74	63.68	61.52	65.19	69.24	65.27	+3.33
Concept+Description Retrieval	subject + concept + description	67.73	64.63	62.23	66.20	70.40	66.24	+4.30

Table 2: Ablation study on Global-MMLU with CA-SFT. Group 1 varies memory alignment quality, Group 2 varies the injected memory content, and Group 3 varies the retrieval query used to access the same memory bank. LRL-Question Retrieval and English-Question Retrieval correspond to direct multilingual retrieval and translate-then-retrieve baselines, respectively. Delta is computed relative to CA-only without memory within each ablation group.

passing CoT (52.50), Cross-CoT (53.94), and Self-Translate (53.38). On MGSM, CA-only reaches 24.93, slightly exceeding SoT (24.00) and outperforming the remaining baselines. This indicates that abstracting fragile LRL inputs into HRL concept definitions can mitigate some elicitation failures more effectively than formatting reasoning steps on unaligned LRL representations or retaining surface-level noise via full-text translation. Appendix H provides setting-matched raw-memory controls to test whether gains come merely from additional retrieved context. These controls use the same CA-SFT checkpoint, retrieval encoder, Top-1 retrieval, threshold, fallback behavior, and decoding setting as CA-Mem, with answer-sanitized retrieved content. On Global-MMLU under CA-SFT, CA-only scores 61.94, while Raw QA Memory,

Raw Trajectory Memory, and Raw QA + Concept Key reach 63.21, 64.03, and 64.49, respectively; CA-Mem reaches 66.24. This shows that raw source-side context and concept-normalized retrieval are helpful, while curated procedural usage summaries provide the strongest gain. The subsequent injection of procedural heuristics provides stable incremental gains. Direct comparison between CA-Mem and CA-only estimates the incremental effect of the Concept Usage Memory. Across all training configurations and benchmarks, CA-Mem consistently improves upon the memory-free CA-only setting. Under CA-SFT, memory augmentation yields a +4.20 point increase on INCLUDE (59.12 vs. 54.92) and a +3.07 point increase on MGSM (28.00 vs. 24.93) over CA-only. These consistent improvements indicate that

while concept activation supports knowledge routing, supplying concept-aligned procedural experience remains important for executing domain-specific logic.

Bootstrap tests in Appendix H support the reliability of the Global-MMLU and INCLUDE gains. On MGSM, the CA-Mem over CA-only gain is positive but marginal, while comparisons against SoT and Self-Translate remain statistically reliable.

5.2 Ablation Study

Table 2 isolates the architectural contributions of memory quality, content fields, and retrieval strategies on Global-MMLU under the CA-SFT configuration. We examine whether improvements derive from semantic alignment or mere context lengthening. Injecting randomly sampled, unrelated memory yields no improvement (61.82, -0.12), while concept-mismatched memory actively degrades reasoning accuracy (61.35, -0.59). Only concept-aligned retrieved memory achieves a clear gain (66.24, $+4.30$). This suggests that reasoning benefits primarily from aligned experiential knowledge, whereas unaligned contexts introduce interference rather than useful information. Evaluating the internal fields of the memory node reveals that the procedural usage summary constitutes the primary performance driver. Providing the declarative *description* alone offers a marginal gain ($+1.18$), whereas isolating the usage summary yields a larger $+3.24$ improvement. The full memory node aggregates both to achieve the maximum $+4.30$ increment. This substantiates our hypothesis that procedural heuristics—encoding application boundaries and conditions—are more effective in this setting than declarative definitions for guiding LRL deduction. Group 3 compares different retrieval-query formulations under the same memory bank. LRL-Question Retrieval uses the raw multilingual question as the retrieval query, corresponding to direct multilingual retrieval, and yields only a $+0.87$ gain over CA-only. English-Question Retrieval first translates or rephrases the question into English before retrieval, corresponding to a translate-then-retrieve baseline, and improves the macro average by $+2.60$. Both remain below Concept-Only Retrieval ($+3.33$) and Concept+Description Retrieval ($+4.30$), suggesting that CA-Mem benefits not merely from English retrieval, but from compact, answer-sanitized concept anchors that remove question-level noise before memory matching. Appendix H further analyzes memory safety: among

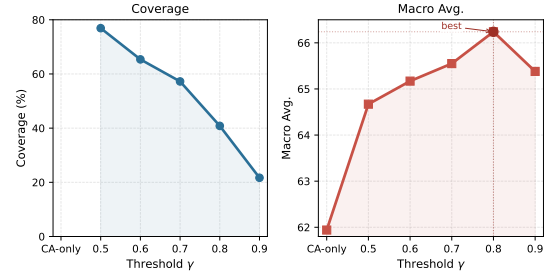


Figure 2: Threshold sensitivity on Global-MMLU with CA-SFT. Coverage denotes the percentage of test examples that receive memory injection.

100 sampled CA-Mem failures, retrieval mismatch accounts for only 12%, while concept activation and final reasoning errors dominate. Appendix I shows that CA-Mem rescues 3,251 CA-only errors while introducing 230 new errors, suggesting that memory injection mostly corrects rather than broadly disrupts predictions.

5.3 Continual Memory Growth

Table 3 evaluates reasoning accuracy as the external memory bank accumulates source-side experiences. Using subject-stratified sampling on Global-MMLU, increasing the memory snapshot size from 0% to 100% of the curated memory bank yields a monotonic performance increase, lifting the Macro Average from 61.94 to 66.24. The marginal gains exhibit a distinct structural pattern. The initial 25% snapshot drives the largest absolute improvement ($+2.04$). This indicates that the early memory snapshot rapidly captures high-frequency, highly reusable procedural heuristics. Subsequent expansions (to 50%, 75%, and 100%) provide diminishing yet positive returns ($+1.09$, $+0.55$, $+0.62$). These later updates primarily assimilate long-tail concepts and fine-grained, subject-specific application boundaries. The consistent upward trajectory supports the continual memory growth paradigm: systematically accumulating abstracted procedural experiences provides scalable enhancements for cross-lingual knowledge elicitation.

5.4 Sensitivity Analysis

Figure 2 shows a clear trade-off between memory precision and injection coverage as the retrieval confidence threshold γ changes. A permissive threshold ($\gamma = 0.5$) triggers memory augmentation for 76.91% of the test instances but yields a sub-optimal Macro Average (64.67) due to the incorporation of loosely aligned, noisy heuristics. Tight-

Source Stream Processed	Memory Snapshot	bn	sw	te	ne	hi	Macro Avg.
0%	CA-only / No Memory	63.61	60.28	58.05	61.71	66.03	61.94
25%	Memory 25%	65.63	62.44	60.11	63.76	67.94	63.98
50%	Memory 50%	66.62	63.81	60.98	64.94	68.99	65.07
75%	Memory 75%	67.13	64.27	61.43	65.58	69.69	65.62
100%	Final Memory	67.73	64.63	62.23	66.20	70.40	66.24

Table 3: Continual memory growth on Global-MMLU with CA-SFT. Memory snapshots are built from subject-stratified source subsets.

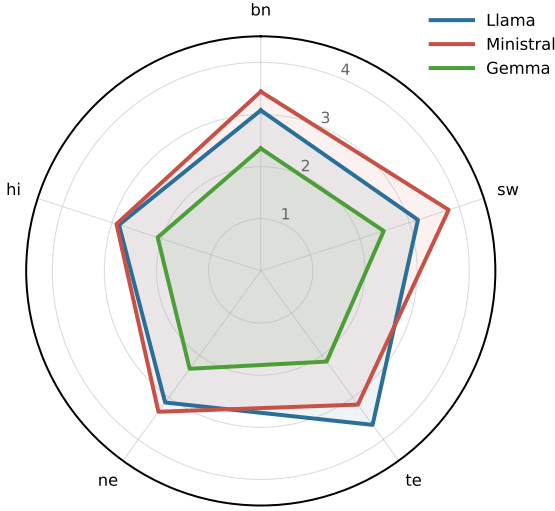


Figure 3: Per-language memory gains of CA-Mem over CA-only across model families on Global-MMLU.

ening the threshold systematically filters this cross-lingual interference, maximizing reasoning accuracy at 66.24 under $\gamma = 0.8$, corresponding to a moderate coverage rate of 40.83%. Conversely, an overly high threshold ($\gamma = 0.9$) restricts coverage to 21.68%, discarding valid procedural knowledge and causing a performance regression to 65.38. This identifies $\gamma = 0.8$ as the best equilibrium in our setting, suggesting that high-precision conceptual matching is preferable to broad but noisy memory injection.

5.5 Cross-Model Family Generalization

The cross-model evaluation in Figure 3 suggests that CA-Mem’s benefits are not tied to a single backbone. Applying the memory-augmented pipeline to Llama-3.1-8B-Instruct (Llama Team, 2024), Ministral-3-8B-Instruct-2512 (Liu et al., 2026a), and Gemma-3-12B-it (Gemma Team, 2025) yields consistent positive increments over the memory-free baseline (CA-only). Specifically, procedural memory injection yields macro-average gains of +3.18, +3.33, and +2.27 across the re-

spective families. The uniform outward expansion in the radar visualization indicates that these gains are broadly observed across all five evaluated LRLs rather than driven by a single linguistic split. This pattern indicates that concept-aligned memory leverages generalized semantic representations across model families.

5.6 Additional Validation

To ensure that the observed gains are not merely caused by adding extra retrieved context, we conduct setting-matched raw-memory controls in Appendix H. Sanitized raw QA memory, raw trajectory memory, and concept-key raw QA memory all underperform curated procedural memory under the same CA-SFT checkpoint, retrieval encoder, Top-1 retrieval, threshold, fallback behavior, and decoding setting. Appendix I further shows that CA-Mem rescues substantially more CA-only errors than it introduces new mistakes, while Appendix G.4 shows that CA-Mem adds only modest overhead over CA-only and remains cheaper than SoT.

6 Conclusion

To overcome the cross-lingual knowledge bottleneck in low-resource reasoning, we introduce the Concept-Anchored Memory Agent. By explicitly abstracting fragile surface inputs into high-resource conceptual definitions, the framework supports more reliable knowledge routing within the pre-trained latent space. This semantic anchor subsequently retrieves procedural heuristics from a curated Concept Usage Memory, mitigating representation mismatch and reducing redundant contexts in standard retrieval-style pipelines. Extensive evaluations across diverse multilingual benchmarks and model architectures indicate that injecting concept-aligned experiential knowledge helps reduce reasoning drift. This methodology offers a scalable direction from passive text conversion to explicit conceptual alignment for multilingual reasoning systems.

Limitations

CA-Mem relies on the quality of the initial concept activation step; when the generated concept is overly broad or incorrect, retrieval may return mismatched procedural memory and provide only limited benefit. Although CA-Mem is designed to support continual memory growth, our evaluation freezes memory snapshots during test-time inference for fair comparison and leakage control; online memory update policies and their stability remain future work. Our current evaluation focuses on multilingual QA and reasoning benchmarks rather than fully interactive tool-use environments, leaving longer-horizon agent tasks and multimodal settings for future study.

References

Zhe Cao, Zhi Qu, Hidetaka Kamigaito, and Taro Watanabe. 2024. Exploring intrinsic language-specific subspaces in fine-tuning multilingual neural machine translation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 21142–21157.

Ding Chen, Simin Niu, Kehang Li, Peng Liu, Xiangping Zheng, Bo Tang, Xinchu Li, Feiyu Xiong, and Zhiyu Li. 2025. Halumem: Evaluating hallucinations in memory systems of agents. *arXiv preprint arXiv:2511.03506*.

Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. *BGE M3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation*. *arXiv preprint arXiv:2402.03216*.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. *QLoRA: Efficient finetuning of quantized LLMs*. In *Advances in Neural Information Processing Systems*, volume 36.

Julen Etxaniz, Gorka Azkune, Aitor Soroa, Oier Lopez de Lacalle, and Mikel Artetxe. 2024. Do multilingual language models think better in english? In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 2: Short Papers)*, pages 550–564.

Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, Haofen Wang, and others. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2(1):32.

Gemma Team. 2025. *Gemma 3 technical report*. *arXiv preprint arXiv:2503.19786*.

Roksana Goworek, Olivia Macmillan-Scott, and Eda B Özyigit. 2025. Bridging language gaps: Advances in cross-lingual information retrieval with multilingual llms. *arXiv preprint arXiv:2510.00908*.

Zhanghao Hu, Qinglin Zhu, Di Liang, Hanqi Yan, Yulan He, and Lin Gui. 2026. Beyond rag for agent memory: Retrieval by decoupling and aggregation. *arXiv preprint arXiv:2602.02007*.

Haoyang Huang, Tianyi Tang, Dongdong Zhang, Xin Zhao, Ting Song, Yan Xia, and Furu Wei. 2023. Not all languages are created equal in llms: Improving multilingual capability by cross-lingual-thought prompting. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 12365–12394.

Minsoo Kim, Victor Bursztyn, Eunye Koh, Shunan Guo, and Seung-won Hwang. 2024. Rada: Retrieval-augmented web agent planning with llms. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 13511–13525.

Alexander H Liu, Kartik Khandelwal, Sandeep Subramanian, Victor Jouault, Abhinav Rastogi, Adrien Sadé, Alan Jeffares, Albert Jiang, Alexandre Cahill, and Alexandre Gavaudan. 2026a. Ministral 3. *arXiv preprint arXiv:2601.08584*.

Chaoqun Liu, Wenxuan Zhang, Yiran Zhao, Luu Anh Tuan, and Lidong Bing. 2025a. *Is translation all you need? a study on solving multilingual tasks with large language models*. In *Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 9594–9614.

Jiaqi Liu, Yaofeng Su, Peng Xia, Siwei Han, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. 2026b. Simplemem: Efficient lifelong memory for llm agents. *arXiv preprint arXiv:2601.02553*.

Jiaqi Liu, Kaiwen Xiong, Peng Xia, Yiyang Zhou, Haonian Ji, Lu Feng, Siwei Han, Mingyu Ding, and Huaxiu Yao. 2025b. Agent0-v1: Exploring self-evolving agent for tool-integrated vision-language reasoning. *arXiv preprint arXiv:2511.19900*.

Llama Team. 2024. *The Llama 3 herd of models*. *arXiv preprint arXiv:2407.21783*.

Edoardo Maria Ponti, Goran Glavaš, Olga Majewska, Qianchu Liu, Ivan Vulić, and Anna Korhonen. 2020. Xcopa: A multilingual dataset for causal common-sense reasoning. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2362–2376.

Rui Qi, Zhibo Man, Yufeng Chen, Fengran Mo, Jinan Xu, and Kaiyu Huang. 2025. *SoT: Structured-of-thought prompting guides multilingual reasoning in*

718	large language models. In <i>Findings of the Association for Computational Linguistics: EMNLP 2025</i> , pages 11024–11039.	775
719		776
720		
721	Jiaxin Qin, Zixuan Zhang, Chi Han, Pengfei Yu, Man-	777
722	ling Li, and Heng Ji. 2024. Why does new knowledge	778
723	create messy ripple effects in llms? In <i>Proceedings</i>	779
724	<i>of the 2024 Conference on Empirical Methods in</i>	780
725	<i>Natural Language Processing</i> , pages 12602–12609.	781
726		782
727	Qwen Team. 2026. <i>Qwen3.5: Accelerating productivity</i>	783
	<i>with native multimodal agents</i> .	
728	Leonardo Ranaldi, Tommaso Tesi, Dirk Hovy, and An-	784
729	dre Freitas. 2025. <i>When natural language is not</i>	785
730	<i>enough: The limits of in-context learning demonstra-</i>	786
731	<i>tions in multilingual reasoning</i> . In <i>Findings of the</i>	787
732	<i>Association for Computational Linguistics: NAACL</i>	
733	<i>2025</i> , pages 7406–7429.	
734	Angelika Romanou, Negar Foroutan, Anna Sotnikova,	
735	Sree Harsha Nelaturu, Shivalika Singh, Rishabh Ma-	
736	heshwary, Micol Altomare, Zeming Chen, Mohamed	
737	Haggag, and Alfonso Amayuelas. 2025. Include:	
738	Evaluating multilingual language understanding with	
739	regional knowledge. In <i>International Conference</i>	
740	<i>on Learning Representations</i> , volume 2025, pages	
741	83291–83322.	
742	Freda Shi, Mirac Suzgun, Markus Freitag, Xuezhi Wang,	
743	Suraj Srivats, Soroush Vosoughi, Hyung Won Chung,	
744	Yi Tay, Sebastian Ruder, Denny Zhou, Dipanjan Das,	
745	and Jason Wei. 2022. Language models are multi-	
746	lingual chain-of-thought reasoners. <i>arXiv preprint</i>	
747	<i>arXiv:2210.03057</i> .	
748	Shivalika Singh, Sagnik Choudhury, Akshay Kedia,	
749	Parag Bhatia, Akshara Prasad, Anirudh Gandhi, Pre-	
750	rana Dsouza, Sunayana Sitaram, Joel Ruben Antony	
751	Moniz, Puneet Bhandari, Amirhossein Abaskohi, Ali	
752	Parsaei, John Donaldson, Rishabh Sharma, Yangling	
753	Li, Kaituo Liu, Marzieh Fadaee, and Sara Hooker.	
754	2025. <i>Global MMLU: Understanding and addressing</i>	
755	<i>cultural and linguistic biases in multilingual evalua-</i>	
756	<i>tion</i> . In <i>Proceedings of the 63rd Annual Meeting of</i>	
757	<i>the Association for Computational Linguistics (Vol-</i>	
758	<i>ume 1: Long Papers)</i> , pages 18761–18799. Associa-	
759	tion for Computational Linguistics.	
760	Tianyi Tang, Wenyang Luo, Haoyang Huang, Dongdong	
761	Zhang, Xiaolei Wang, Wayne Xin Zhao, Furu Wei,	
762	and Ji-Rong Wen. 2024. Language-specific neurons:	
763	The key to multilingual capabilities in large language	
764	models. In <i>Proceedings of the 62nd Annual Meet-</i>	
765	<i>ing of the Association for Computational Linguistics</i>	
766	<i>(Volume 1: Long Papers)</i> , pages 5701–5715.	
767	Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten	
768	Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V Le,	
769	and Denny Zhou. 2022. Chain-of-thought prompting	
770	elicits reasoning in large language models. <i>Advances</i>	
771	<i>in neural information processing systems</i> , 35:24824–	
772	24837.	
773	Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao	
774	Tan, and Yongfeng Zhang. 2026. A-mem: Agentic	
	memory for llm agents. <i>Advances in Neural Informa-</i>	775
	<i>tion Processing Systems</i> , 38:17577–17604.	776
	Weihao Xuan, Rui Yang, Heli Qi, Qingcheng Zeng,	
	Yunze Xiao, Aosong Feng, Dairui Liu, Yun Xing,	
	Junjue Wang, and Fan Gao. 2025. Mmlu-prox: A	
	multilingual benchmark for advanced large language	
	model evaluation. In <i>Proceedings of the 2025 Con-</i>	
	<i>ference on Empirical Methods in Natural Language</i>	
	<i>Processing</i> , pages 1513–1532.	
	Guibin Zhang, Muxin Fu, Guancheng Wan, Miao Yu,	
	Kun Wang, and Shuicheng Yan. 2025a. G-memory:	
	Tracing hierarchical memory for multi-agent systems,	
	2025. <i>arXiv preprint arXiv:2506.07398</i> .	
	Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li,	
	Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong	
	Wen. 2025b. A survey on the memory mechanism of	
	large language model-based agents. <i>ACM Transac-</i>	
	<i>tions on Information Systems</i> , 43(6):1–47.	
	Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and	
	Yanlin Wang. 2024. Memorybank: Enhancing large	
	language models with long-term memory. In <i>Pro-</i>	
	<i>ceedings of the AAAI conference on artificial intelli-</i>	
	<i>gence</i> , volume 38, pages 19724–19731.	
	Wenhao Zhu, Hongyi Liu, Qingxiu Dong, Jingjing Xu,	
	Shujian Huang, Lingpeng Kong, Jiajun Chen, and	
	Lei Li. 2024. Multilingual machine translation with	
	large language models: Empirical results and analy-	
	sis. In <i>Findings of the association for computational</i>	
	<i>linguistics: NAACL 2024</i> , pages 2765–2781.	

Appendix

A Dataset and Evaluation Details

Benchmark	Code	Language	#Examples
Global-MMLU	bn	Bengali	14,042
Global-MMLU	sw	Swahili	14,042
Global-MMLU	te	Telugu	14,042
Global-MMLU	ne	Nepali	14,042
Global-MMLU	hi	Hindi	14,042
Global-MMLU	en	English	14,042
Global-MMLU	zh	Chinese	14,042
INCLUDE	bn	Bengali	548
INCLUDE	te	Telugu	548
INCLUDE	ne	Nepali	500
INCLUDE	hi	Hindi	547
MGSM	en	English	250
MGSM	bn	Bengali	250
MGSM	sw	Swahili	250
MGSM	te	Telugu	250

Table 4: Language-level evaluation sizes for the benchmarks discussed in this paper.

Global-MMLU (Singh et al., 2025) includes Bengali, Swahili, Telugu, Nepali, Hindi, English, and Chinese splits, with 14,042 examples per language. In the main experiments, we report per-language accuracy and Macro Avg. on the five low-resource languages, while retaining the English and Chinese splits for additional generalization tests. INCLUDE (Romanou et al., 2025) evaluates multilingual examination QA on Bengali, Telugu, Nepali, and Hindi, and we report Weighted Avg. because language-specific test sizes differ. MGSM (Shi et al., 2022) evaluates multilingual math word problem solving on Bengali, Swahili, and Telugu, with 250 examples per language, and we report Macro Avg.

We use benchmark-specific answer extraction and normalization rules consistent with the evaluation setup described in the main text.

B Additional Benchmark Results

The gains on MGSM are smaller than those on Global-MMLU and INCLUDE. This pattern is consistent with MGSM consisting of math word problems, where structured decomposition methods such as SoT naturally align with the required multi-step arithmetic reasoning. Nevertheless, CA-Mem still improves over SoT under CA-SFT, indicating that concept-aligned procedural memory remains complementary even when the task format already favors structured reasoning.

C Training and Implementation Details

Table 6 summarizes the fine-tuning, retrieval, and decoding settings used in the experiments.

Artifact use and licenses. We use publicly available benchmarks and model artifacts only for research evaluation and method development, following their released licenses or usage terms. The used artifacts include Global-MMLU, INCLUDE, MGSM, MMLU-ProX, Qwen3.5-9B, BGE-M3, Llama, Ministral, and Gemma. We do not redistribute benchmark test sets, evaluation answers, or proprietary model outputs. The constructed memory entries are answer-sanitized and exclude source questions, options, answer labels, gold answers, correct options, and answer-bearing shortcuts.

D Prompt Templates

D.1 Prompt Templates for Concept Distillation, CA-SFT, and Evaluation

The prompt templates below are written in the Global-MMLU format, where each input consists of a question and labeled answer options. INCLUDE is also a multiple-choice benchmark and uses the same prompt format. MGSM is instantiated with the same method-specific prompts but without answer options; its final answer field is the normalized final answer rather than an option label.

For methods that construct an intermediate representation, we use a two-stage evaluation protocol. The first stage generates the method-specific intermediate representation without producing the final answer. The second stage predicts the answer from the intermediate representation and the benchmark input fields required by that method. For Cross-CoT, SoT, CA-only, and CA-Mem, the original question and options are retained in the second stage; for Self-Translate, the second stage uses the translated English question and options. For CA-Mem, memory retrieval is performed between the two stages: after concept activation, we construct the retrieval query by concatenating the subject, activated concept, and concept description. The retrieved usage summary is then provided to the second-stage answer prompt. The retrieved memory is not generated by the LLM.

All methods are evaluated only by the final answer field.

D.2 Concept Distillation and CA-SFT Prompts

Teacher Prompt for Concept Distillation

Task: concept_distillation

Language: <language>

Subject: <subject>

You are given a multilingual multiple-choice question with answer options.

Model	Method	INCLUDE					MGSM			
		bn	te	ne	hi	W. Avg.	bn	sw	te	M. Avg.
Base	Zero-shot	49.82	37.04	48.60	57.40	48.20	17.20	16.00	8.80	14.00
	CoT	52.19	37.23	48.80	58.68	49.23	20.80	20.80	13.20	18.27
	Cross-CoT	54.01	37.41	49.00	59.60	50.02	23.60	23.20	16.00	20.93
	Self-Translate	56.02	37.41	46.80	56.67	49.28	24.80	22.80	15.60	21.07
	SoT	53.83	37.41	49.20	57.59	49.51	25.20	25.60	18.00	22.93
	CA-only	55.47	37.59	48.80	60.69	50.67	24.80	25.60	17.60	22.67
	CA-Mem	59.85	42.88	53.60	62.89	54.83	27.60	28.00	20.80	25.47
Direct-SFT	Zero-shot	49.27	38.87	49.60	56.67	48.58	18.00	16.80	11.20	15.33
	CoT	51.64	39.23	50.40	58.32	49.88	22.00	21.20	15.60	19.60
	Cross-CoT	52.55	39.42	51.00	59.05	50.49	24.00	23.60	17.20	21.60
	Self-Translate	54.01	40.33	52.60	59.41	51.56	25.20	23.60	16.40	21.73
	SoT	52.19	39.78	52.60	59.41	50.96	25.60	25.20	18.80	23.20
	CA-only	54.74	40.88	53.20	60.15	52.22	26.40	25.60	19.60	23.87
	CA-Mem	59.12	44.89	56.40	63.44	55.95	29.20	28.40	22.40	26.67
CA-SFT	Zero-shot	51.46	39.05	51.00	59.41	50.21	20.00	17.20	12.00	16.40
	CoT	54.01	40.88	53.40	61.79	52.50	23.20	21.60	16.40	20.40
	Cross-CoT	55.47	42.34	55.00	63.07	53.94	25.60	23.20	18.00	22.27
	Self-Translate	56.75	41.24	52.60	62.89	53.38	25.60	24.40	16.80	22.27
	SoT	54.93	42.52	56.80	63.44	54.37	26.80	25.60	19.60	24.00
	CA-only	56.38	43.80	55.60	63.99	54.92	28.00	26.40	20.40	24.93
	CA-Mem	60.95	47.63	60.40	67.64	59.12	30.80	29.60	23.60	28.00

Table 5: Full INCLUDE and MGSM results in a single merged table. CA-Mem denotes CA-only with concept usage memory.

Item	Value
Base model	Qwen3.5-9B (Qwen Team, 2026)
Fine-tuning	4-bit QLoRA (Dettmers et al., 2023)
LoRA rank	64
LoRA alpha	128
LoRA dropout	0.05
Max sequence length	2048
Batch size	2 per GPU
Gradient accumulation	16
Learning rate	2e-4
Epochs	2
Scheduler	cosine
Retrieval encoder	BAAI/bge-m3 (Chen et al., 2024)
Decoding temperature	0

Table 6: Implementation details used in the current setup.

884	Task. Produce a high-quality supervision tuple for concept-anchored reasoning. You must output three fields. concept: a concise English concept cue that captures the key knowledge, rule, or reasoning skill required by the question. description: a short English definition of the concept that explains it generally and is reusable for similar questions. answer: the correct option label. Requirements for concept. Must be written in English; concise and informative; domain-appropriate for the subject; centered on the key knowledge or reasoning operation; not a generic label such as “reasoning” or “multiple-choice”; not contain the answer label; not directly reveal the correct option; reusable beyond this question. Requirements for description. Must be written in English; define the general concept, rule, or reasoning skill; describe when or how the concept is used; not provide a step-by-step solution to the current question; not mention the correct option; not copy long spans from the question or options; avoid overly broad or vague wording. Requirements for answer. Must be exactly one option label from the provided options. Self-check before returning. Verify that the answer is consistent with the question and options, the concept is necessary for solving the question, and the description is reusable and free of answer leakage.
	Return a valid JSON object only: { <pre>"concept": "...", "description": "...", "answer": "..." }</pre> Question: <question> Options: <options> CA-SFT Prompt for Concept Generation Task: generate_concept Language: <language> Subject: <subject> You are given a multilingual multiple-choice question.

887	Task. Extract a reusable English concept cue and a short English concept description. Requirements for concept. Must be written in English; concise and informative; domain-appropriate for the subject; capture the key knowledge, rule, or reasoning skill required by the question; not be a generic label such as “reasoning,” “knowledge,” or “multiple-choice”; not contain the answer label; not directly reveal the correct option. Requirements for description. Must be written in English; define or explain the concept in general terms; be reusable for similar questions; not solve the current question step by step; not mention the correct option; not copy long spans from the question or options.
	Return a valid JSON object only: { <pre>"concept": "...", "description": "..." }</pre> Question: <question> Options: <options> CA-SFT Prompt for Concept-Grounded Answering Task: answer_with_concept Language: <language> Subject: <subject> You are given: 1. a multilingual multiple-choice question; 2. an activated English concept; 3. a short English description of the concept. Task. Use the concept and description as auxiliary guidance to select the single best answer. Rules. The question and options are authoritative. The concept and description are auxiliary cues, not replacements for the question. If the concept or description is partially mismatched, rely on the question and options. Do not force the question to fit the concept. Do not invent missing facts. Do not provide explanation. The final answer must be one of the option labels exactly as given. Return a valid JSON object only: { <pre>"answer": "..." }</pre> Activated concept: <concept> Concept description: <description> Question: <question> Options: <options> LLM Judge Prompt for Concept Filtering Task: concept_quality_judgment You are evaluating whether a distilled concept-description pair is useful for solving a multilingual multiple-choice question. Input. Language: <language> Subject: <subject> Question:

887

888

889

890

891

892

893

<question> Options: <options> Gold answer: <gold_answer>
Teacher answer: <teacher_answer> Distilled concept: <concept>
Distilled description: <description>
Scoring scale. For relevance, specificity, description_quality, and reusability: 1 = poor, 2 = weak, 3 = acceptable, 4 = good, 5 = excellent. For risk: 1 = very low risk, 2 = low risk, 3 = moderate risk, 4 = high risk, 5 = very high risk. The risk direction is reversed: higher risk is worse.
Criteria. relevance: whether the concept captures the knowledge, rule, or reasoning skill needed to answer the question. specificity: whether the concept is informative rather than overly generic. description_quality: whether the description correctly and clearly defines the concept in general terms. reusability: whether the concept and description can be reused beyond this exact instance. risk: whether the concept or description is misleading, answer-specific, shortcut-inducing, overfit to the current question, mismatched, or likely to harm downstream reasoning.
Important. Do not reward a concept merely because the teacher answer is correct. Penalize answer leakage, option-specific shortcuts, vague concepts such as “basic knowledge,” and descriptions that solve the instance step by step instead of defining reusable knowledge.

Return a valid JSON object only: {
"relevance": ...,
"specificity": ...,
"description_quality": ...,
"reusability": ...,
"risk": ...,
"rationale": "..."
}

D.3 Memory Curation Prompt

Memory Curator Prompt for Concept Usage Memory

Task: memory_curation

Language: <language>

Subject: <subject>

You are given a source-side solving trajectory produced by a teacher model and evaluated by an external verifier. Your task is to curate an answer-sanitized concept usage memory node from the solving trajectory and verifier feedback.

The memory node should capture reusable procedural knowledge about how to apply the activated concept. It must not preserve the original question, option text, answer label, gold answer, or any answer-bearing shortcut.

Task. Construct a reusable memory node with the following fields: subject, concept, description, and usage_summary.

The usage_summary must be organized into three procedural components: trigger_invariants, decision_rules, and negative_error_boundaries.

Definitions. trigger_invariants describe when the concept should be activated, including stable cues, conditions, or problem patterns that indicate the concept is relevant.

decision_rules describe how to apply the concept once it is activated, including reusable reasoning operations, comparisons, constraints, or calculation principles.

negative_error_boundaries describe what should not be assumed or what mistakes should be avoided, including common confusions, invalid shortcuts, distractor patterns, or boundary cases.

Requirements. The memory node must be written in English. It must describe reusable procedural guidance rather than a solution to the current instance. It must be reusable for future questions in the same subject area. It must not copy the source question. It must not copy any option text. It must not mention the model’s final answer label. It must not mention the gold answer. It must not reveal the correct option. It must not include answer-bearing shortcuts. It must not store a worked solution to the source question. It must avoid source-specific entities, numbers, or surface details unless they are necessary to define the general concept.

Quality control. Set keep to true only if the trajectory is verifier-approved and the memory node is general, reusable, answer-sanitized, and aligned with the activated concept. Set keep to false if the trajectory is not verifier-approved, the concept is too vague, the usage summary leaks the answer, copies the instance, or cannot be reused.

Return a valid JSON object only: {
"keep": true,
"subject": "...",
"concept": "...",
"description": "...",
"usage_summary": {
"trigger_invariants": "...",
"decision_rules": "...",
"negative_error_boundaries": "..."
},
"drop_reason": ""
}

If the memory node should be discarded, return: {
"keep": false,
"subject": "...",
"concept": "...",
"description": "...",
"usage_summary": {
"trigger_invariants": "",
"decision_rules": "",
"negative_error_boundaries": ""
},
"drop_reason": "..."
}

Question: <question>

Options: <options>

Activated concept: <concept>

Concept description: <description>

Teacher solving trajectory: <solution_trace>

Teacher prediction: <teacher_prediction>

Verification status: <verification_status>

Verifier feedback: <verifier_feedback>

D.4 Evaluation-Time Prompt Templates

Zero-Shot Evaluation Prompt

Method: Zero-shot

Language: <language>

Subject: <subject>

You are given a multiple-choice question in <language>.

Task. Select the single best answer from the provided options.

Rules. Read the question and all options carefully. Do not provide explanation. Do not invent information not supported by the question or options. The answer must be one of the option labels exactly as given. Preserve the original option label format.

Return a valid JSON object only: {

"answer": "..."

}

Question: <question>

Options: <options>

CoT Evaluation Prompt

Method: Chain-of-Thought

Language: <language>

Subject: <subject>

You are given a multiple-choice question in <language>.

Task. Reason step by step, then select the single best answer.

Rules. Reason in English when possible. Preserve all quantities, entities, units, negations, and option labels. Do not ignore any condition in the question. Do not invent information not supported by the question or options. The final answer must be one of the option labels exactly as given.

Return a valid JSON object only: {

"reasoning": "...",

"answer": "..."

}

Question: <question>

Options: <options>

D.5 Two-Stage Baseline Prompts

Cross-CoT Stage 1

Method: Cross-CoT, Stage 1

Language: <language>

Subject: <subject>

You are given a multilingual multiple-choice question.

Task. Understand the question in its original language and produce a high-resource-language reasoning trace in English.

Rules. Reason in English. Preserve all numbers, units, named entities, negations, comparisons, and option labels. Do not mechanically translate the entire question word by word. Focus on the knowledge, constraints, and reasoning steps needed to solve the question. Do not output the final answer label. Do not state “the answer is ...” Do not invent information not supported by the question or options.

Return a valid JSON object only: {

"english_reasoning": "..."

}

Question: <question>

Options: <options>

Cross-CoT Stage 2

Method: Cross-CoT, Stage 2

Language: <language>

Subject: <subject>

You are given: 1. the original multilingual multiple-choice question; 2. the original answer options; 3. an English reasoning trace produced from the question.

912	Task. Select the single best answer using the question, options, and English reasoning trace. Rules. The original question and options are authoritative. Use the English reasoning trace as auxiliary guidance. If the reasoning trace appears partially incorrect, incomplete, or mismatched, rely on the original question and options. Do not provide explanation. The final answer must be one of the option labels exactly as given. Preserve the original option label format. Return a valid JSON object only: { "answer": "..." } English reasoning: <english_reasoning> Question: <question> Options: <options>
	Self-Translate Stage 1 Method: Self-Translate, Stage 1 Language: <language> Translate or faithfully rephrase the following multiple-choice question into English.
913	Rules. Do not answer the question. Do not explain. Preserve all numbers, units, formulas, named entities, negations, and option labels. Keep the original option structure. Translate the question and options faithfully. If a term should not be translated, preserve it. Return a valid JSON object only: { "english_question": "...", "english_options": "..." } Question: <question> Options: <options>
	Self-Translate Stage 2 Method: Self-Translate, Stage 2 Language: English Subject: <subject> You are given an English translation or rephrasing of a multilingual multiple-choice question.
914	Task. Select the single best answer from the translated English question and options. Rules. Use only the translated question and options. Preserve the original option labels exactly. Do not provide explanation. Do not invent information not supported by the question or options. The final answer must be one of the option labels exactly as given. Return a valid JSON object only: { "answer": "..." } Translated question: <english_question> Translated options: <english_options>
	SoT Stage 1 Method: SoT, Stage 1 Language: <language> Subject: <subject> You are given a multilingual multiple-choice question.
915	Task. Convert the problem into a compact structured representation for reasoning. Steps. Identify the essential meaning of the question. Extract key entities, quantities, units, conditions, relations, and the target variable. Preserve negations, comparisons, constraints, and option labels. Represent the problem in a language-agnostic structured form. Rules. Do not output the final answer label. Do not state "the answer is ...". Do not solve the question step by step. Do not invent facts not present in the question or options. The structure should be concise but sufficient for answering. Preserve the original option labels exactly.
	Return a valid JSON object only: { "structure": { "question_intent": "...", "entities": [...], "conditions": [...], "relations": [...], "target": "...", "option_map": {"<label>": "..."} } } Question: <question> Options: <options>
916	SoT Stage 2 Method: SoT, Stage 2 Language: <language> Subject: <subject> You are given: 1. the original multilingual multiple-choice question; 2. the original answer options; 3. a structured representation of the problem.
	Task. Select the single best answer using the structured representation.

Rules. The original question and options are authoritative. Use the structure as auxiliary guidance. If the structure is incomplete, incorrect, or partially mismatched, rely on the original question and options. Check all constraints, negations, quantities, units, and option labels before answering. Do not provide explanation. The final answer must be one of the option labels exactly as given. Return a valid JSON object only: { "answer": "..." } Structured representation: <structure> Question: <question> Options: <options>

D.6 CA-Mem Evaluation Prompts

CA-only / CA-Mem Stage 1: Concept Activation Method: CA-only / CA-Mem, Stage 1 Language: <language> Subject: <subject> You are given a multilingual multiple-choice question.
Task. Activate the core English concept needed to solve the question. The goal is not to translate the full question. The goal is to identify the high-level concept, rule, or reasoning skill required by the question. Requirements for concept. Must be written in English; concise and informative; domain-appropriate for the given subject; capture the central knowledge or reasoning operation needed by the question; not be a generic label such as "reasoning," "knowledge," or "multiple-choice"; not contain the answer label; not directly reveal the correct option. Requirements for description. Must be written in English; define the concept in general terms; describe when or how the concept is used; not solve the specific question step by step; not mention the correct option; not copy long spans from the question or options. Return a valid JSON object only: { "concept": "...", "description": "..." } Question: <question> Options: <options>
CA-only Stage 2: Concept-Grounded Answering Method: CA-only, Stage 2 Language: <language> Subject: <subject> You are given: 1. a multilingual multiple-choice question; 2. an activated English concept; 3. a short English description of the concept.
Task. Use the concept and description as auxiliary guidance to select the single best answer. Rules. The question and options are authoritative. The concept and description are auxiliary cues, not replacements for the question. If the concept or description is partially mismatched, rely on the question and options. Do not force the question to fit the concept. Do not invent missing facts. Do not provide explanation. The final answer must be one of the option labels exactly as given. Return a valid JSON object only: { "answer": "..." } Activated concept: <concept> Concept description: <description> Question: <question> Options: <options>
CA-Mem Retrieval Step After Stage 1, the retrieved memory is not generated by the LLM. We construct the retrieval query by concatenating the subject, activated concept, and concept description: <pre>retrieval_query = concat(subject, concept, description)</pre> The query is encoded with the retrieval encoder and matched against the frozen concept usage memory bank. We retrieve the Top-1 memory node only when its similarity score is at least γ. If no memory node passes the threshold, CA-Mem falls back to CA-only.
CA-Mem Stage 2: Memory-Augmented Answering Method: CA-Mem, Stage 2 Language: <language> Subject: <subject> You are given: 1. a multilingual multiple-choice question; 2. an activated English concept and description; 3. a retrieved concept usage memory. The retrieved memory is obtained from the frozen concept usage memory bank using the subject, activated concept, and concept description as the retrieval query. It contains procedural guidance about how the concept is usually applied and may include trigger conditions, decision rules, and common pitfalls.
Task. Use the concept, description, and retrieved memory as auxiliary

guidance to select the single best answer.

Rules. The question and options are authoritative. The concept and description are auxiliary cues, not replacements for the question. Use the retrieved memory only if it is relevant to the activated concept and the current question. If the memory is partially mismatched, ignore the mismatched part. Do not copy an answer from memory. Do not assume that the memory contains the answer to this question. Use the memory to guide concept application, not to replace the question. Check whether any option violates the concept boundary or a known pitfall. Do not provide explanation. The final answer must be one of the option labels exactly as given.

```
Return a valid JSON object only: {
  "answer": "..."}
}
Activated concept: <concept>
Concept description: <description>
Retrieved usage memory: <usage_summary>
Question: <question>
Options: <options>
```

D.7 MGSM Adapter

MGSM Adapter

For MGSM, we instantiate the same method-specific prompts without the Options field, since MGSM is evaluated as open-ended math question answering. The final answer field contains the normalized final answer rather than an option label. Intermediate representations, such as English reasoning traces, structured representations, concept-description pairs, and retrieved memory summaries, follow the same method-specific templates.

D.8 Raw-Memory Baseline Prompts

Raw-Memory Baseline Stage 2

Method: Raw-memory baseline

Language: <language>

Subject: <subject>

You are given: 1. a multilingual multiple-choice question; 2. a retrieved raw memory snippet.

The retrieved memory snippet may come from sanitized raw QA memory, sanitized raw trajectory memory, or raw QA memory indexed by a concept key. Use it as auxiliary guidance only if it is relevant to the question. The question and options are authoritative. Do not copy an answer from memory. Do not assume the memory contains the correct option. Do not provide explanation. The final answer must be one of the option labels exactly as given.

```
Return a valid JSON object only: {
  "answer": "..."}
}
Retrieved content: <retrieved_content>
Question: <question>
Options: <options>
```

D.9 Answer Extraction

Final Answer Extraction

Given a model response, extract the final benchmark answer in the required normalized format. Preserve the option label exactly for multiple-choice tasks and the normalized final answer for MGSM. Do not add explanation.

E Leakage Prevention and Memory Construction

E.1 Leakage Prevention

Before constructing either SFT data or memory nodes, we apply instance-level decontamination between the MMLU-ProX (Xuan et al., 2025) source pool and all evaluation benchmarks, including Global-MMLU, INCLUDE, and MGSM. We remove exact duplicates, normalized duplicates, and answer-equivalent source–test reformulations before both CA-SFT construction and memory curation. We canonicalize each question with Unicode normalization, whitespace normalization, punctuation normalization, lowercasing where applicable, and removal of option-label artifacts. This filtering

is instance-level rather than concept-level: we do not remove source examples merely because they share high-level concepts or reasoning skills with evaluation examples, since concept-level transfer is the intended setting. The final memory bank is frozen before evaluation, and test examples only perform read-only retrieval. Memory nodes are answer-sanitized and exclude source questions, option text, answer labels, gold answers, correct-option text, and answer-bearing shortcuts.

E.2 SFT Data Construction

For supervised training, we start from the duplicate-filtered MMLU-ProX (Xuan et al., 2025) source pool and apply teacher-correctness filtering followed by an LLM-based concept-quality filter. We retain 4,700 rows for each of the five training languages. Each retained row is converted into two CA-SFT instances: concept generation and concept-grounded answering. Direct-SFT uses the same retained rows but keeps only answer supervision. The filtering criteria for the retained CA-SFT rows are shown in Table 7.

Filter	Criterion
Teacher correctness	teacher_answer == gold
Parsing validity	valid concept, description, and answer
Risk control	risk ≤ 3
Overall quality	mean_score ≥ 3.25
Relevance	relevance ≥ 3
Description quality	description_quality ≥ 3

Table 7: Filtering criteria for the main_keep split used for concept distillation and CA-SFT construction.

The main_keep criteria above are used for concept distillation and CA-SFT construction. Memory construction uses the same duplicate-filtered source pool but retains only verifier-approved teacher trajectories before curator abstraction. mean_score is computed as the average of relevance, specificity, description_quality, and reusability. The risk score is not included in mean_score and is used only as a filtering constraint. The borderline bucket is retained only as a fallback pool; the final 4,700-per-language setting uses the main_keep split exclusively.

E.3 Memory Construction

Memory construction uses the same duplicate-filtered source pool but follows a separate trajectory-based curation pipeline. A teacher model first solves filtered source-side questions and produces solving trajectories. In our implementation, GPT-5.4 is used as the teacher model. An external verifier uses the source-side gold label only to

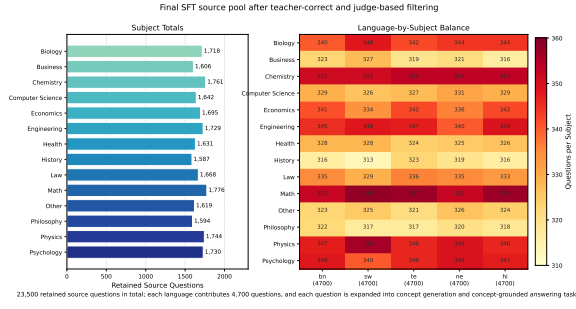


Figure 4: Subject distribution of the final main_keep source pool used to build the SFT data. The teacher first produces {concept, description, answer} in a single response, we keep only teacher-correct samples, and then apply judge-based quality filtering. The final retained CA-SFT pool is evenly balanced across the five training languages, with 4,700 rows per language. Each retained row is later expanded into two CA-SFT instances: concept generation and concept-grounded answering.

determine whether a trajectory is eligible for curation. Only verifier-approved teacher trajectories are passed to the memory curator. The source-side gold label is not provided as an input field to the memory curator and is not stored in the memory node. From the verified trajectory pool, we construct 6,934 candidate memory nodes and retain 5,087 final concept memory nodes after curation. Each retained node stores an answer-sanitized usage summary with three components: trigger invariants, decision rules, and negative error boundaries. The final memory bank is frozen during evaluation and excludes original source questions, evaluation questions, option text, answer labels, gold answers, correct option text, and answer-bearing shortcuts. The memory curator prompt, provided in Appendix D, converts verifier-approved teacher trajectories into answer-sanitized usage summaries with three fields: trigger invariants, decision rules, and negative error boundaries.

F Memory Bank Statistics

This section summarizes the frozen memory bank used in the final experiments. Table 8 reports the overall memory-bank size, Table 9 gives the category distribution, and Table 10 shows how category counts grow as more source data is processed.

Statistic	Value
Candidate memory nodes	6,934
Final concept memory nodes	5,087
Coarse categories	14

Table 8: Memory bank statistics.

Category	Nodes	Ratio	Category	Nodes	Ratio
Math	473	9.30%	Law	366	7.19%
Physics	428	8.41%	Economics	342	6.72%
Chemistry	417	8.20%	Psychology	329	6.47%
Biology	406	7.98%	Business	317	6.23%
Health	394	7.75%	History	304	5.98%
Computer Science	383	7.53%	Philosophy	281	5.52%
Engineering	357	7.02%	Other	290	5.70%
Total	5,087	100%			

Table 9: Category distribution of the final memory bank.

Category	25%	50%	75%	100%
Math	118	237	355	473
Physics	107	214	321	428
Chemistry	104	209	313	417
Biology	102	203	305	406
Health	99	197	296	394
Computer Science	96	192	287	383
Engineering	89	179	268	357
Law	92	183	275	366
Economics	86	171	257	342
Psychology	82	165	247	329
Business	79	159	238	317
History	76	152	228	304
Philosophy	70	141	211	281
Other	72	145	218	290
Total	1,272	2,547	3,819	5,087

Table 10: Category-wise continual memory growth across processed source fractions from 25% to 100%.

G Additional Ablation and Sensitivity Results

This section reports supplementary ablations and sensitivity analyses on Global-MMLU, covering memory construction, memory growth, retrieval sensitivity, thresholding, injected examples, and cross-model generalization.

G.1 Memory Construction Pipeline

Table 11 reports the memory construction pipeline ablation.

This ablation evaluates whether the memory construction pipeline improves over raw candidate memories. Raw candidate memory already improves CA-only, indicating that source-side solving traces contain transferable concept usage experience. Curator refinement further improves performance by reducing noise and redundancy, while minimal update achieves the best result by preserving stable memory nodes and only applying necessary updates.

G.2 Top-k Sensitivity

Table 12 reports the sensitivity to the number of retrieved memory nodes.

Top-1 retrieval performs best, while adding Top-2 and Top-3 nodes slightly reduces performance. This suggests that the most aligned memory node contains the useful procedural signal, whereas ad-

Method	Candidate	Curator	Update	bn	sw	te	ne	hi	Macro Avg.
CA-only	—	—	—	63.61	60.28	58.05	61.71	66.03	61.94
Raw Candidate Memory	yes	no	append raw	67.03	64.07	61.35	65.64	69.62	65.54
Curated Memory	yes	yes	full rewrite	67.42	64.34	61.97	65.98	70.19	65.98
Curated Minimal Update	yes	yes	minimal update	67.73	64.63	62.23	66.20	70.40	66.24

Table 11: Memory construction pipeline ablation on Global-MMLU with CA-SFT. All memory variants are built from the filtered MMLU-ProX (Xuan et al., 2025) source pool.

Retrieved Nodes	bn	sw	te	ne	hi	Macro Avg.
Top-1	67.73	64.63	62.23	66.20	70.40	66.24
Top-2	67.41	64.31	61.91	65.88	70.08	65.92
Top-3	67.10	64.00	61.60	65.57	69.77	65.61

Table 12: Top-k retrieval sensitivity on Global-MMLU with CA-SFT.

ditional nodes can introduce adjacent but noisy concept usage summaries.

G.3 Rationale for Top-1 Memory Retrieval

CA-Mem uses Top-1 retrieval with confidence gating rather than injecting multiple retrieved memories. This design follows the role of Concept Usage Memory: each memory node is intended to provide a compact procedural cue for applying a specific activated concept, rather than serving as a collection of factual evidence passages. Injecting multiple memories can introduce competing heuristics, redundant descriptions, or partially mismatched boundary conditions, especially when retrieved nodes are close in embedding space but correspond to different concept usages.

We therefore prioritize high-precision memory injection over broad recall. If the top retrieved node does not pass the confidence threshold, CA-Mem falls back to the memory-free CA-only setting instead of forcing potentially noisy memory into the prompt. This design is consistent with the threshold sensitivity analysis, where overly permissive retrieval increases coverage but introduces noisier memory, while overly strict retrieval removes useful procedural cues.

G.4 Inference Token Cost

Table 13 reports inference token cost under the CA-SFT setting. As expected, methods with intermediate representations require more tokens than single-pass prompting. CA-Mem incurs higher token usage than Zero-shot because it performs concept activation before memory-augmented answering. However, its overhead over CA-only is modest: the average total token count increases from 1016.66 to 1046.78 tokens per example. This indicates that the main inference overhead comes from the concept-activation stage, while gated memory injection adds only a small amount of additional

input context. Compared with SoT, CA-Mem uses fewer total tokens and substantially fewer output tokens, suggesting that compact concept-aligned procedural memory can provide strong accuracy gains without relying on long generated rationales.

G.5 Anchor Language Analysis

Table 14 reports an anchor-language sanity check on Global-MMLU with Qwen3.5 Base. We compare the default English-anchor setting with a Chinese-anchor variant. In the Chinese-anchor variant, the model uses Chinese concept-description anchors; when memory is enabled, retrieval is performed with Chinese concept-description memory keys. This analysis tests whether CA-Mem benefits merely from moving the query into a high-resource language space, or whether the standardized English anchor space used by the main memory bank provides a more stable retrieval interface.

Chinese anchoring is competitive on Qwen3.5 Base, improving CA-only from 53.87 to 57.24 Macro Avg. This suggests that Qwen-style backbones can effectively use Chinese as a high-resource conceptual interface. However, adding concept usage memory under the Chinese-anchor setting still improves performance, raising the Macro Avg. from 57.24 to 58.29. This indicates that procedural memory remains complementary to concept anchoring even when the anchor language changes.

The competitive performance of Chinese anchoring on Qwen3.5 Base supports the general principle underlying CA-Mem: the key mechanism is projecting fragile LRL inputs into any dense, well-trained high-resource concept space, rather than relying on a language-specific property of English. Both English and Chinese serve as effective conceptual interfaces because they share a common characteristic: substantially higher pre-training exposure compared to the evaluated low-resource

Method	Avg. input	Avg. output	Total	Rel. cost	Acc.
Zero-shot	384.84	6.00	390.84	1.00×	55.48
CoT	407.84	54.24	462.08	1.18×	59.07
Cross-CoT	914.06	51.42	965.48	2.47×	60.52
Self-Translate	731.34	91.50	822.84	2.11×	61.67
SoT	1103.56	164.74	1268.30	3.25×	61.22
CA-only	976.52	40.14	1016.66	2.60×	61.94
CA-Mem	1006.74	40.04	1046.78	2.68×	66.24

Table 13: Inference token cost and accuracy on Global-MMLU under the CA-SFT setting. Token counts are averaged per example over the full evaluation set and include all inference stages required by each method. Relative cost is computed against Zero-shot.

Method	Anchor Lang.	Memory Key Lang.	Memory	bn	sw	te	ne	hi	Macro Avg.
CA-only	English	–	No	56.67	56.00	46.08	52.81	57.80	53.87
CA-only-ZH	Chinese	–	No	59.44	59.27	49.58	55.79	62.11	57.24
CA-Mem	English	English	Yes	60.77	60.37	50.10	56.83	61.66	57.95
CA-Mem-ZH	Chinese	Chinese	Yes	60.42	60.31	50.61	56.78	63.32	58.29

Table 14: Anchor-language analysis on Global-MMLU using Qwen3.5 Base. English-anchor rows correspond to the Base setting in Table 1. Chinese-anchor rows use Chinese concept-description anchors; CA-Mem-ZH uses Chinese concept-description memory keys for retrieval.

languages. The main experiments standardize on English anchoring for a practical reason: the concept usage memory bank, retrieval keys, and cross-model evaluation pipeline are all organized around English concept-description pairs, enabling consistent comparison across Qwen, Llama, Ministral, and Gemma backbones. Models such as Qwen3.5 that have strong Chinese pre-training may leverage Chinese anchoring with comparable effectiveness, but a unified English anchor space remains preferable for cross-model reproducibility.

G.6 Threshold Sensitivity by Language

Table 15 reports the per-language threshold sensitivity.

We run threshold sensitivity on Global-MMLU with CA-SFT using the final frozen memory bank, which contains 5,087 concept-level memory nodes. Each node contains only the concept, coarse-grained subject/category, description, and usage summary, and does not contain original questions, option text, or answer information.

For each test instance, CA-Mem first generates an English concept and description using CA-only, then retrieves Top-1 memory using the subject, generated concept, and generated description. Memory is injected only if the Top-1 similarity score is at least γ . Table 15 and Figure 2 show the threshold sensitivity on Global-MMLU. As γ increases from 0.5 to 0.8, coverage decreases from 76.91% to 40.83%, while Macro Avg. improves from 64.67 to 66.24, indicating that filtering lower-confidence matches improves memory precision. When γ is further increased to 0.9, coverage drops to 21.68%

and Macro Avg. decreases to 65.38. This suggests that an overly high filtering threshold discards useful memories, although the remaining high-confidence matches still outperform CA-only. We therefore use $\gamma = 0.8$ as the development-selected threshold in the main experiments.

G.7 High-resource English Results

Figure 5 reports the high-resource English results.

CA-Mem also improves over CA-only in English, but the gains are smaller than in low-resource languages. This is consistent with the hypothesis that English inputs already activate internal concept knowledge more reliably, leaving less room for concept routing improvements.

G.8 Memory Injection Breakdown

Table 16 reports the accuracy breakdown by memory-injection status.

At the development-selected threshold $\gamma = 0.8$, 28,667 out of 70,210 Global-MMLU test examples receive memory injection, corresponding to 40.83% of the test set. Table 16 shows that the overall gain is concentrated in this high-confidence memory-injected subset. On the memory-injected examples, CA-Mem improves over CA-only by 10.54 points, whereas fallback cases remain identical to CA-only by construction.

G.9 Cross-model Family Generalization

This subsection examines whether the CA-Mem gains observed for the main model also transfer to other model families with different multilingual

Threshold γ	Coverage	bn	sw	te	ne	hi	Macro Avg.
CA-only	—	63.61	60.28	58.05	61.71	66.03	61.94
0.5	76.91%	66.31	63.08	60.63	64.50	68.83	64.67
0.6	65.38%	66.83	63.61	61.08	65.02	69.30	65.17
0.7	57.24%	67.18	64.01	61.45	65.49	69.61	65.55
0.8	40.83%	67.73	64.63	62.23	66.20	70.40	66.24
0.9	21.68%	66.84	63.93	61.50	65.34	69.27	65.38

Table 15: Threshold sensitivity on Global-MMLU with CA-SFT. Coverage denotes the percentage of test examples that receive memory injection. The default setting $\gamma = 0.8$ matches the main CA-Mem result in Table 1.

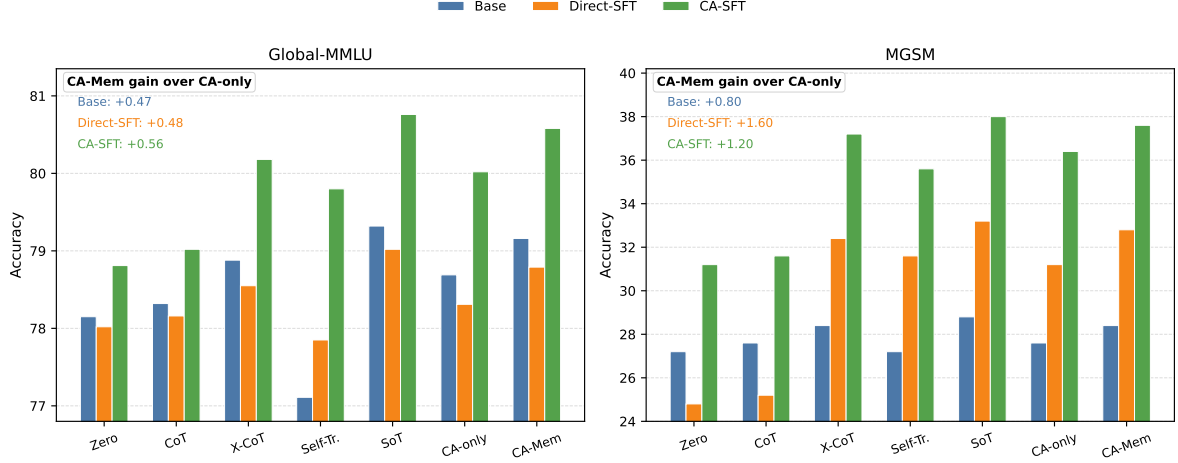


Figure 5: English results on Global-MMLU and MGSM. Each subplot compares prompting, concept-anchored, and memory-augmented methods across Base, Direct-SFT, and CA-SFT settings. Numbers above the CA-Mem bars indicate the absolute gain over CA-only.

capabilities. Table 17 reports cross-model generalization across three model families.

Table 17 shows that CA-Mem generalizes consistently across all three model families. Relative to CA-only, the average gain is +3.18 for Llama-3.1-8B-Instruct, +3.33 for Ministral-3-8B-Instruct-2512, and +2.27 for Gemma-3-12B-it. The improvement is also broad-based across languages rather than concentrated in a single split: every model family shows positive gains on bn, sw, te, ne, and hi. Notably, the Llama-3.1-8B results also show that CoT is relatively weak compared with concept-anchored variants. This pattern supports our motivation that simply eliciting longer reasoning chains does not necessarily correct the initial cross-lingual concept routing error; when the initial concept is misactivated, CoT can elaborate an incorrect reasoning path rather than recover the intended knowledge. The corresponding radar visualization is shown in Figure 3 in the main text.

H Additional Validation and Safety Analysis

This section collects validation analyses that are not part of the main result tables: raw-memory baselines, development-set threshold selection, memory

safety checks, and manual failure analysis.

H.1 Statistical Reliability

We estimate 95% confidence intervals with paired bootstrap resampling over evaluation examples. For each comparison, we resample examples with replacement, recompute the aggregate metric, and report the distribution of paired score differences. Global-MMLU and MGSM use macro-average differences, while INCLUDE uses weighted-average differences. We report two-sided paired bootstrap p -values.

Table 18 shows that the main Global-MMLU and INCLUDE improvements are statistically reliable across comparisons. MGSM exhibits wider confidence intervals due to its smaller evaluation size; the gain over CA-only is positive but marginal, while comparisons against SoT and Self-Translate remain statistically reliable.

H.2 Raw Memory Baselines

Table 19 reports the Global-MMLU CA-SFT raw-memory baseline protocol. All methods use the same CA-SFT checkpoint, the same retrieval encoder, Top-1 retrieval, the same retrieval threshold, the same fallback behavior, and temperature 0. The

Group	#Examples	CA-only Correct	CA-Mem Correct	CA-only Acc.	CA-Mem Acc.	Gain
Memory injected	28,667	18,467	21,488	64.42	74.96	+10.54
No-memory fallback	41,543	25,018	25,018	60.22	60.22	+0.00
Total	70,210	43,485	46,506	61.94	66.24	+4.30

Table 16: Accuracy breakdown by memory-injection status on Global-MMLU with CA-SFT.

Model	Method	bn	sw	te	ne	hi	Avg.
Llama-3.1-8B-Instruct	Zero-shot	40.56	41.61	39.14	41.63	45.80	41.75
	CoT	39.62	38.97	19.62	39.04	45.52	36.55
	CA-only	44.99	43.25	42.19	44.18	48.00	44.52
	CA-Mem	48.07	46.42	45.84	47.30	50.86	47.70
Ministral-3-8B-Instruct-2512	Zero-shot	53.30	40.54	50.30	51.61	55.24	50.20
	CoT	56.54	38.77	52.51	54.32	60.16	52.46
	CA-only	56.87	45.23	55.39	56.27	59.27	54.61
	CA-Mem	60.31	49.02	58.56	59.61	62.18	57.94
Gemma-3-12B-it	Zero-shot	58.64	54.75	58.90	59.59	61.34	58.64
	CoT	59.98	56.37	60.13	60.45	61.73	59.73
	CA-only	60.85	56.57	61.73	60.82	62.77	60.55
	CA-Mem	63.20	59.05	63.88	63.14	64.85	62.82

Table 17: Cross-model family generalization on Global-MMLU. CA-Mem denotes CA-only with concept usage memory.

raw QA and raw trajectory memory variants are answer-sanitized before retrieval and do not expose gold answers, answer labels, correct option text, or answer-bearing shortcuts.

The raw-memory variants improve over CA-only, indicating that source-side examples and trajectories contain transferable signals. However, both sanitized raw QA memory and sanitized raw trajectory memory remain below CA-Mem. Raw QA + Concept Key further improves over English-question retrieval, showing that concept-normalized retrieval keys are beneficial even for raw memory. The best performance is achieved by CA-Mem, suggesting that curated procedural usage summaries provide less noisy and more actionable guidance than raw retrieved context.

H.3 Development-Set Threshold Selection

The default retrieval threshold $\gamma = 0.8$ is selected on a held-out development split. This development split is not used for CA-SFT training or memory construction. After selecting $\gamma = 0.8$, we keep this value fixed for all test evaluations.

As γ increases, memory coverage decreases while memory precision improves. The development results peak at $\gamma = 0.8$, which provides the best balance between accuracy and coverage. We therefore fix $\gamma = 0.8$ before test evaluation rather than selecting the threshold on test results.

H.4 Leakage Control and Memory Safety Audit

Table 21 summarizes the leakage-control and memory-safety audit.

The following memory-node examples show only sanitized reusable fields. They omit source

questions, options, answer labels, gold answers, and correct option text.

Subject: law

Concept: Obscenity regulation

Description: Government may regulate obscene material, but protected speech boundaries require applying the relevant doctrinal criteria rather than treating all adult content as automatically unprotected.

trigger_invariants: Activate this concept when a legal question concerns restrictions on allegedly obscene expression, public distribution, private access, or the boundary between protected and unprotected speech.

decision_rules: Identify whether the rule targets obscene material under the governing doctrinal test, then check the context of access, distribution, and statutory scope before deciding whether regulation is permissible.

negative_error_boundaries: Do not assume that any adult-oriented material is automatically regulable; avoid collapsing private or limited-access contexts into public distribution.

Subject: biology

Concept: Endosymbiotic theory

Description: The endosymbiotic theory explains the origin of mitochondria and chloroplasts through ancestral symbiotic relationships between cells.

trigger_invariants: Activate this concept when a biology question asks about organelle origins, mitochondria, chloroplasts, bacterial-like features, membranes, DNA, or evolutionary evidence for symbiosis.

decision_rules: Compare each candidate claim against stable evidence for endosymbiosis, such as organelle autonomy, genetic material, replication patterns, and bacterial-like structural features.

negative_error_boundaries: Do not treat generic organelle function as evidence for origin; avoid confusing current cellular roles with evolutionary support for the theory.

H.5 Manual Failure Analysis

We sample 100 incorrect Global-MMLU cases from the CA-SFT + CA-Mem setting, using an

Benchmark	Metric	Comparison	Diff.	95% CI	<i>p</i> -value
Global-MMLU	Macro Avg.	CA-Mem vs. CA-only	+4.30	[+3.98, +4.64]	<0.001
Global-MMLU	Macro Avg.	CA-Mem vs. SoT	+5.02	[+4.67, +5.36]	<0.001
Global-MMLU	Macro Avg.	CA-Mem vs. Self-Translate	+4.57	[+4.24, +4.91]	<0.001
INCLUDE	Weighted Avg.	CA-Mem vs. CA-only	+4.20	[+2.29, +6.07]	<0.001
INCLUDE	Weighted Avg.	CA-Mem vs. SoT	+4.75	[+2.85, +6.67]	<0.001
INCLUDE	Weighted Avg.	CA-Mem vs. Self-Translate	+5.74	[+3.78, +7.75]	<0.001
MGSM	Macro Avg.	CA-Mem vs. CA-only	+3.07	[-0.13, +6.27]	0.060
MGSM	Macro Avg.	CA-Mem vs. SoT	+4.00	[+0.80, +7.20]	0.016
MGSM	Macro Avg.	CA-Mem vs. Self-Translate	+5.73	[+2.40, +8.93]	<0.001

Table 18: Paired bootstrap confidence intervals for key CA-SFT comparisons. Global-MMLU and MGSM use macro-average differences, while INCLUDE uses weighted-average differences. We report two-sided paired bootstrap *p*-values.

Method	Retrieval Key	Retrieved Content	bn	sw	te	ne	hi	Macro Avg.
CA-only	-	-	63.61	60.28	58.05	61.71	66.03	61.94
Raw QA Memory	English question	sanitized raw QA memory	64.58	61.50	59.32	63.18	67.45	63.21
Raw Trajectory Memory	English question	sanitized raw solving trajectory	65.38	62.36	60.14	64.07	68.21	64.03
Raw QA + Concept Key	subject + generated concept + generated description	sanitized raw QA memory	65.72	62.91	60.61	64.52	68.67	64.49
CA-Mem	subject + generated concept + generated description	curated concept usage memory	67.73	64.63	62.23	66.20	70.40	66.24

Table 19: Raw-memory baselines on Global-MMLU with CA-SFT. All methods use the same CA-SFT checkpoint, retrieval encoder, Top-1 retrieval, threshold, fallback behavior, and decoding setting. Raw QA and raw trajectory memories are answer-sanitized before retrieval.

γ	Dev Coverage	Dev Accuracy
0.5	75.80%	64.35
0.6	65.20%	65.02
0.7	53.10%	65.71
0.8	41.40%	66.08
0.9	22.30%	65.54

Table 20: Development-set threshold selection. The threshold $\gamma = 0.8$ achieves the best development accuracy and is fixed before all test evaluations. Coverage denotes the percentage of examples receiving memory injection.

approximately stratified sample across bn, sw, te, ne, and hi where possible. Each error is assigned one primary failure type: concept activation error, retrieval mismatch, insufficient procedural guidance, reasoning / execution error, or benchmark or translation ambiguity.

The largest remaining failure source is concept activation error, where the generated English concept is overly broad or misaligned with the question. Reasoning and execution errors are also frequent, especially for questions requiring multi-step computation, fine-grained option comparison, or exact symbolic manipulation. Retrieval mismatch accounts for a smaller portion of failures, suggesting that gated concept-based retrieval usually avoids injecting unrelated memory once the upstream concept is correctly activated. These results indicate that CA-Mem primarily depends on reliable concept activation and final reasoning quality, rather than retrieval alone.

I Prediction Flip Analysis

This section provides the per-language breakdown of prediction flips between CA-only and CA-Mem in Table 23. A prediction flip analysis further shows that CA-Mem rescues 3,251 CA-only errors while introducing only 230 new errors, yielding a net gain of 3,021 additional correct predictions.

J Qualitative Case Studies

This section presents answer-sanitized qualitative cases selected from prediction flips and retrieval diagnostics. To avoid benchmark leakage, we omit gold labels, option texts, and direct answer-bearing content. We include success, failure, and contrastive cases to illustrate when concept anchoring and concept usage memory help, and when upstream concept errors or retrieval mismatch still limit CA-Mem.

J.1 Case A: Memory Repairs Concept Usage

Language / Subject: bn / law

Case Type: Success — CA-only wrong, CA-Mem correct

Question gist: The question concerns whether a motel owner can be prosecuted for showing adult films privately inside motel rooms, under a statute that prohibits obscene films in places open to the public.

Activated concept: Obscenity regulation

Retrieved memory:

The retrieved memory is shown in sanitized form and does not include source question text, option text, or answer labels.

Risk	Control / Audit Check	Status
Source-test overlap	Remove exact, normalized, and answer-equivalent source-test duplicates before SFT and memory construction	Filtered
Answer leakage	Exclude gold labels, answer labels, correct-option text, and answer-bearing shortcuts from memory nodes	Controlled
Raw-instance memorization	Exclude source questions, option text, and raw QA content; retain only sanitized concept descriptions and usage summaries	Controlled
Test-time contamination	Freeze the memory bank before evaluation; test examples only perform read-only retrieval	Controlled

Table 21: Leakage-control and memory-safety audit.

Failure Type	Count	Proportion
Concept activation error	34	34%
Retrieval mismatch	12	12%
Insufficient procedural guidance	18	18%
Reasoning / execution error	28	28%
Benchmark or translation ambiguity	8	8%
Total	100	100%

Table 22: Manual failure analysis of 100 randomly sampled CA-Mem errors on Global-MMLU under CA-SFT.

Lang	Both C.	Resc.	Hurt	Both W.	Net	Resc.	Hurt
bn	8,890	621	42	4,489	+579	12.16	0.47
sw	8,417	658	48	4,919	+610	11.80	0.57
te	8,097	641	54	5,250	+587	10.88	0.66
ne	8,619	677	46	4,700	+631	12.58	0.53
hi	9,232	654	40	4,116	+614	13.71	0.43
Total	43,255	3,251	230	23,474	+3,021	12.16	0.53

Table 23: Prediction flip analysis of CA-Mem relative to CA-only on Global-MMLU with CA-SFT.

Concept: Obscenity regulation

Subject: law

Description: Government may ban obscene material, but ordinary speech is protected unless it meets strict obscenity criteria.

Sanitized usage cue: Before applying a general obscenity regulation, check the doctrinal boundary of the rule; distinguish public distribution from private or limited-access contexts.

Outcome: CA-only predicts incorrectly, whereas CA-Mem uses the retrieved procedural cue and answers correctly.

Interpretation: This case shows that CA-only can activate the right concept but still apply it too broadly. The model correctly identifies the relevant doctrine as *Obscenity regulation*, but CA-only treats the general power to regulate obscene material as sufficient for upholding the prosecution. CA-Mem retrieves a memory node that does not contain an answer cue, but instead reminds the model to check the boundary condition of the doctrine, especially the distinction between public distribution and private or limited-access viewing. This procedural cue repairs the concept-usage error and flips the final prediction to the correct answer.

Takeaway: CA-only helps the model identify what concept is relevant, while CA-Mem helps the model determine how to apply that concept under the specific constraints of the question.

J.2 Declarative vs. Procedural Memory Contrast

To further clarify the distinction between declarative and procedural memory, we revisit Case A under the same answer-sanitized diagnostic setting. The retrieved *description* provides a general definition of the activated concept, whereas the *usage summary* specifies how the concept should be applied under boundary conditions.

Declarative description only: Government may ban obscene material, but ordinary speech is protected unless it meets strict obscenity criteria.

Procedural usage summary: Before applying a general obscenity regulation, check the doctrinal boundary of the rule; distinguish public distribution from private or limited-access contexts.

The description-only cue identifies the broad legal doctrine but does not specify which contextual distinction should control the decision. In contrast, the procedural usage summary highlights the relevant boundary condition and instructs the model to avoid over-applying the general rule. This illustrates why concept usage memory is not merely a declarative concept definition: it provides an application procedure for using the activated concept under task-specific constraints. This qualitative pattern is consistent with the memory-content ablation in Table 2, where *Usage Summary Only* yields a larger gain than *Description Only*.

J.3 Case B: Concept Anchoring Prevents Cross-Lingual Retrieval Drift

Language / Subject: sw / biology

Case Type: Success — low-resource retrieval drift corrected by concept anchoring

Question gist: The item asks which statement does not support the endosymbiotic theory of the origin of mitochondria and chloroplasts.

Activated concept: Endosymbiotic theory

Retrieval comparison:

Without anchoring: *Prejudgment attachment* (law; mismatched retrieval)

With concept anchoring: *Endosymbiotic theory* (biology; aligned retrieval)

Outcome: Direct raw-question retrieval drifts to an unrelated law memory, whereas concept-anchored

Table 24: Summary of answer-sanitized qualitative cases.

Case	Type	Subject	Main Insight
A	Success	law	Memory repairs concept usage
B	Success	biology	Anchoring prevents retrieval drift
C	Failure	economics	Wrong concept activates wrong memory frame
D	Contrastive failure	physics	Non-anchored retrieval drifts cross-domain

retrieval returns the aligned biology memory and yields the correct prediction.

Interpretation: This case illustrates why CA-Mem does not retrieve memory directly with the raw low-resource-language question. The drift from a biology problem to an unrelated law memory indicates a representation mismatch between the low-resource surface form and the English concept-level memory bank. After anchoring through the generated concept and description, retrieval becomes concept-aligned and the final prediction becomes correct.

Takeaway: Concept anchoring stabilizes cross-lingual retrieval. It converts a noisy low-resource-language question into a high-resource concept key that can reliably access the intended memory node.

J.4 Case C: Wrong Concept Activation Leads Memory to the Wrong Frame

Language / Subject: bn / economics

Case Type: Failure — generated concept is wrong

Question gist: The question asks how money demand changes when nominal GDP increases.

Activated concept: Interest rate effect

Error source: Concept activation error.

Retrieved memory:

Concept: Interest rate effect

Subject: economics

Description: The interest rate effect is the change in money demand caused by changes in the interest rate.

Outcome: CA-only and CA-Mem both remain incorrect because retrieval follows the mistaken concept anchor instead of the intended economic mechanism.

Interpretation: This failure occurs at the concept activation stage. The underlying question is about nominal GDP and transaction demand for money, not primarily about an interest-rate shift. Because retrieval is conditioned on the generated concept, CA-Mem faithfully follows the wrong conceptual frame and cannot recover the answer. Retrieval therefore reinforces the mistaken reasoning path instead of redirecting it.

Takeaway: CA-Mem is not an oracle retriever. Its effectiveness depends on the quality of upstream concept activation. When the generated concept is wrong, memory can reinforce the wrong reasoning frame instead of correcting it.

J.5 Case D: Retrieval Drift without Concept Anchoring

Language / Subject: bn / physics

Case Type: Contrastive failure — non-anchored retrieval drift

Question gist: The item asks for the order of element layers formed around a star as stellar evolution progresses, starting from the outer layer.

Activated concept: Not applicable in this non-anchored contrast.

Non-anchored retrieval result:

Non-anchored retrieved memory: *Ignorance of refutation* (philosophy; mismatched retrieval)

Outcome: Retrieval drifts to an irrelevant philosophy memory and fails before concept-aligned reasoning can be established. This is a contrastive failure case for non-anchored retrieval, not the final CA-Mem configuration.

Interpretation: In this case, the underlying physics question concerns stellar nucleosynthesis and the sequence of element layers formed during stellar evolution. Without concept anchoring, retrieval can still return an unrelated philosophy memory that is procedurally unhelpful for the target concept. We therefore present this case as a retrieval-boundary contrast rather than as a failure of the final anchored CA-Mem system.

Takeaway: CA-Mem requires both a useful concept anchor and precise memory retrieval. The anchored subject + generated concept + generated description query helps prevent this kind of cross-domain retrieval drift.